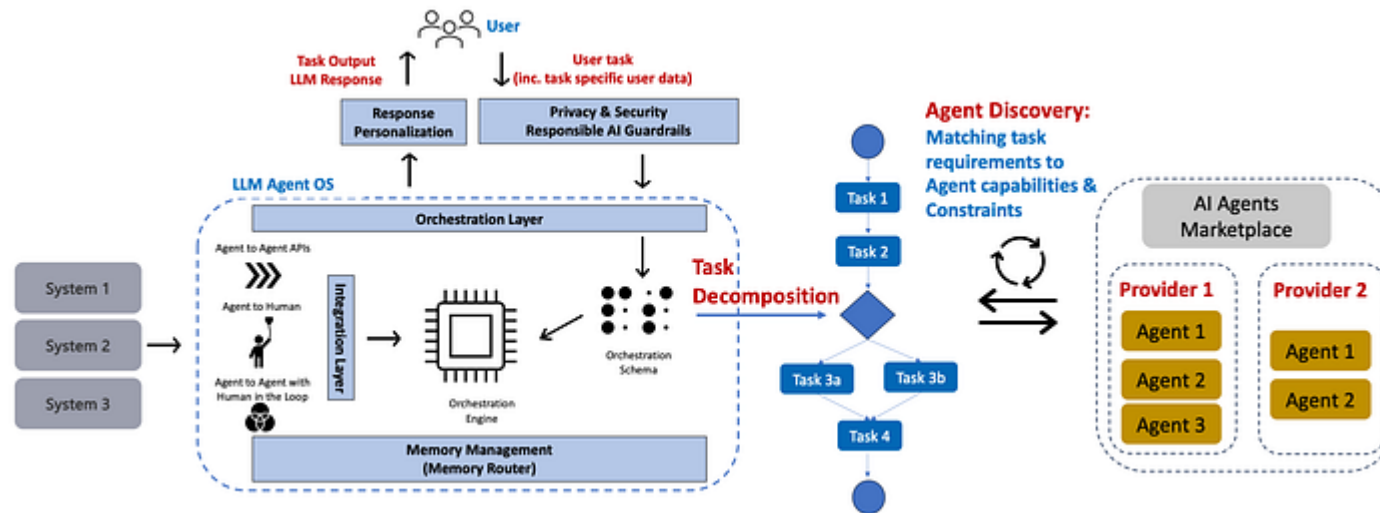


[< Go to the original](#)

# Agentic AI MCP Tools Governance

Agentic AI discovery and governance guidelines for the Enterprise



Debmalya Biswas

Follow

a11y-light · September 16, 2025 (Updated: September 16, 2025) · Free: No

## 1. Problem Statement

responses, AI agents can execute complex tasks autonomously, e.g., make a sale, plan a trip, make a flight booking, book a contractor to do a house job, order a pizza. Fig. 1 below illustrates the evolution of agentic AI systems.

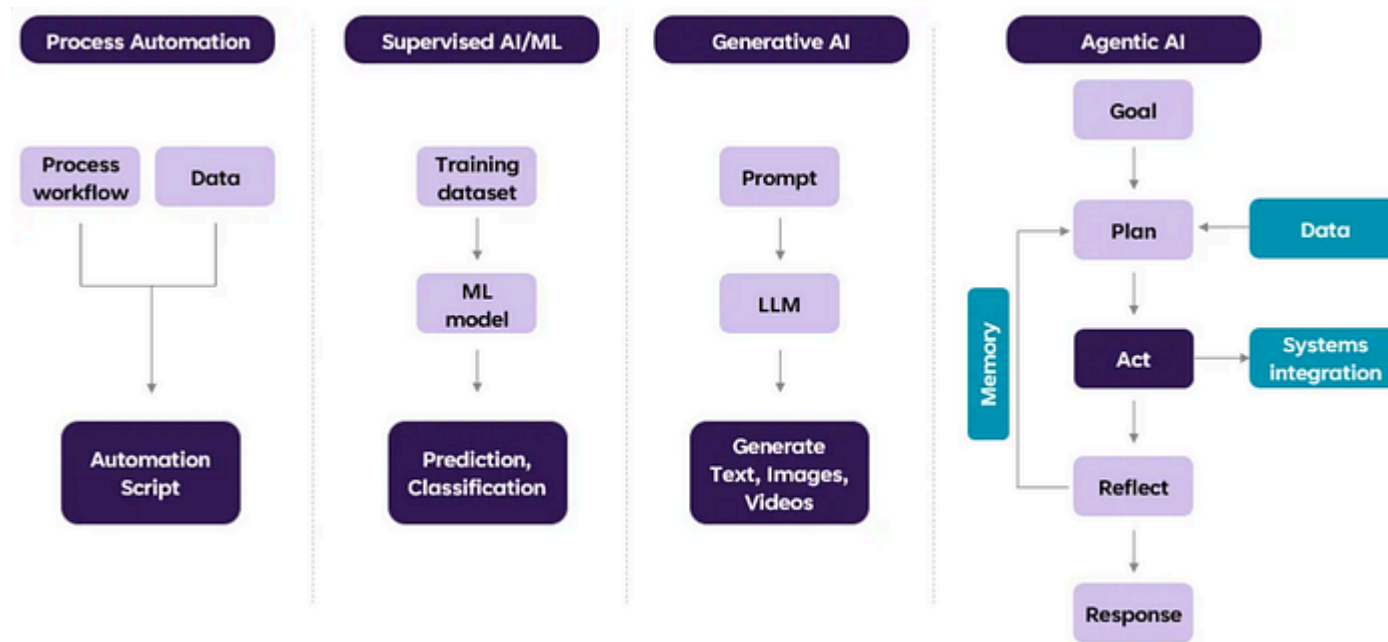


Fig. 1: Agentic AI evolution (Image by Author)

Bill Gates recently envisioned a future where we would have an AI agent that is able to process and respond to natural language and accomplish a number of different tasks. Gates used planning a trip as an example.

use its knowledge of your preferences to book and purchase those things on your behalf.

The key characteristics of agentic AI systems are their autonomy and reasoning prowess that allow them to decompose complex tasks into smaller executable tasks, and then orchestrate their execution with integrated (external) **tools** in a way that can monitor, reflect, and self-correct the execution as and when needed. Given this,

agentic AI has the potential to disrupt almost every business process prevalent in an enterprise today.

Unfortunately, this rapid adoption of AI agents in the enterprise is happening in an ungoverned fashion, and we are starting to see glimpses of the agentic chaos that we have all been dreading. This is true, esp. for tools, where model context protocol (**MCP**) has heralded the rise of hundreds of tools in the enterprise without any standardization. For example,

- HuggingFace has started serving its Spaces apps over MCP
- Composio exposes hundreds of tools over 100+ managed MCP services

So the availability of agentic tools is skyrocketing, but this ungoverned rise is actually leading to more confusion and less productivity.

For example, OpenAI recommends having "fewer than 20 functions at any one time"

Keep the number of functions small for higher accuracy. Evaluate your performance with different numbers of functions. Aim for fewer than 20 functions at any one time, though this is just a soft suggestion.

Too many overlapping tools also leads to the problem of **discoverability**. In a previous work, I have stressed the need for a more descriptive formalism to describe the capabilities and limitations of agents / tools.

The problem is illustrated in Fig. 2 (adapted from LangGraph multi-agent workflows illustration) where tool discovery today relies on natural language descriptions of tools. Given this, how do we expect an algorithm (or human for that matter) to select the right tool for say a web search among tools described ambiguously as follows (actual tools taken from the Smithery catalog):

**Freedium**[ko-fi](#) [librepay](#) [patreon](#)

---

- `web_search`

- `ai-web-search`
- `batch-web-search`
- `answer_query_websearch`
- `get-web-search-ex`

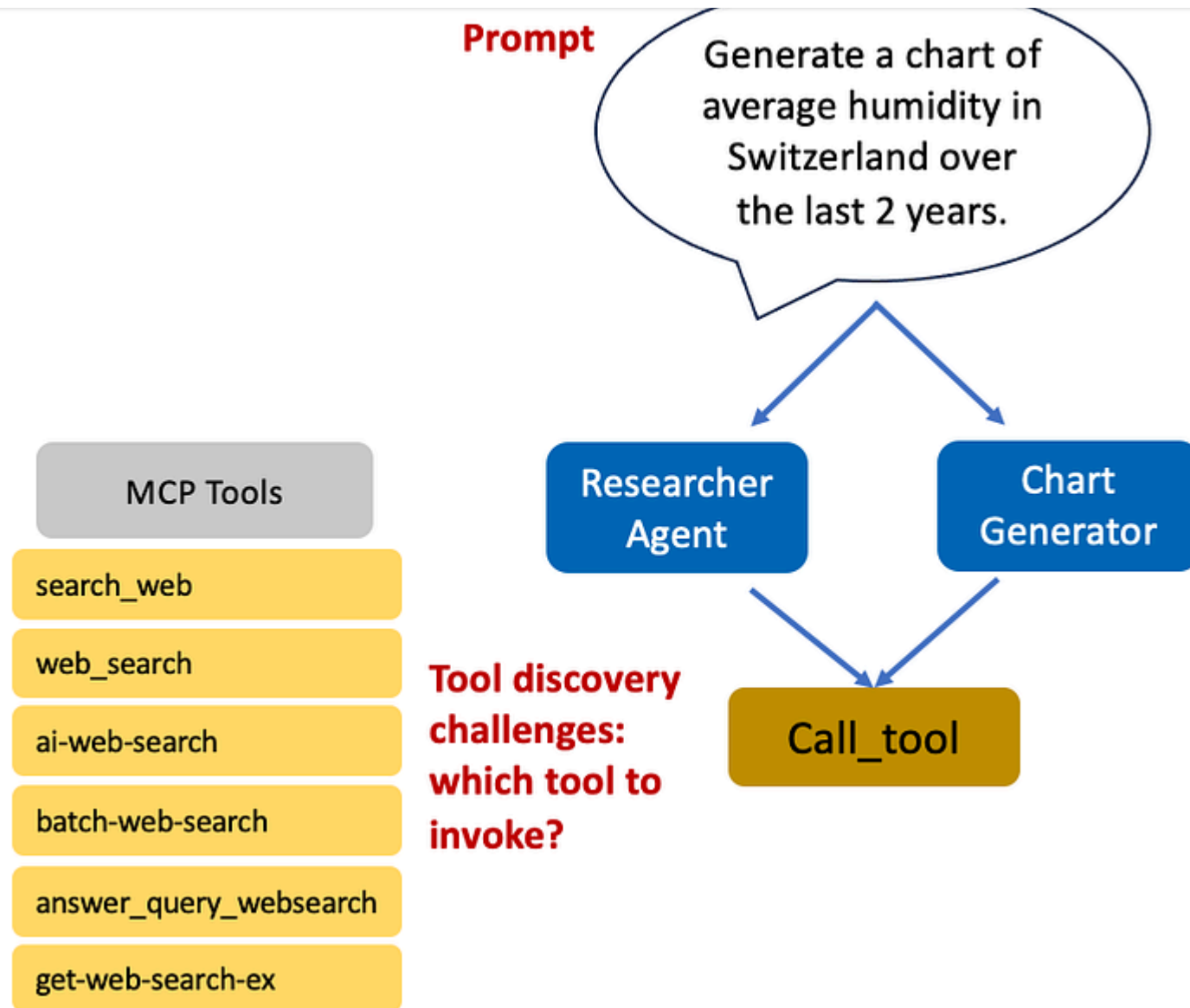


Fig. 2: Agentic AI tool discovery challenges (Image by Author)

deep dive into the discovery aspect of agents / tools in Section 3, highlighting the challenges of optimal agent / tool discovery based on their textual descriptions.

To address these challenges:

- we first outline a learning-to-rank (L2R) algorithm (Section 3.1) to identify the **top-k** agent(s) based on natural language descriptions of both the tasks and marketplace agents.
- we further outline a **constraints** based formalism to capture / specify the capabilities and limitations of services provided by an AI agent in Section 3.2 — to address the ambiguity and lack of semantic information during discovery.

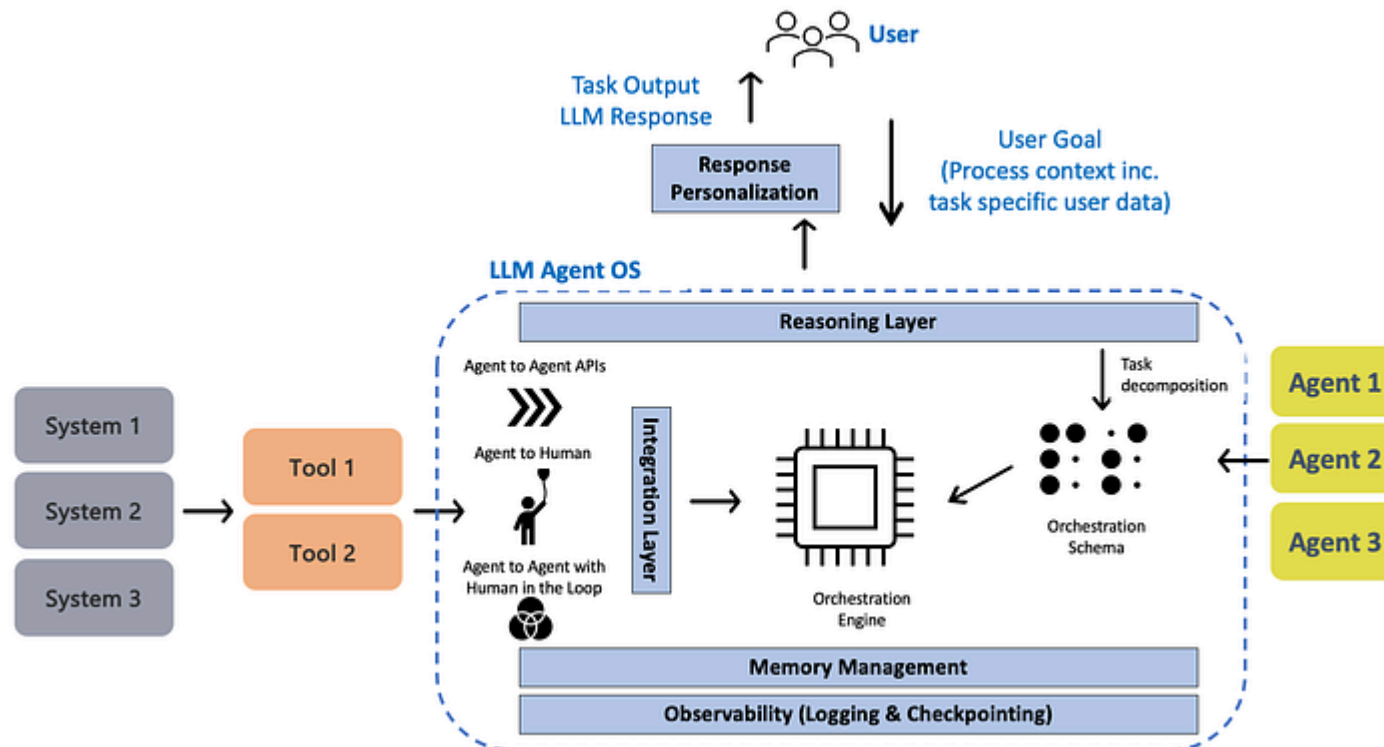
Finally, we outline some best practices and guidelines in Section 4 towards deploying MCP tools in a governed fashion in the enterprise.

## 2. Agentic AI Reference Architecture

In this section, we outline the key modules of a reference agentic AI platform — illustrated in Fig. 3:

- **Reasoning** module: to decompose complex tasks and adapt their execution to achieve the given objective;

- **Orchestration module:** to orchestrate and monitor (observe) the execution of multi-agent systems;
- **Integration module:** MCP tools to integrate with enterprise systems, e.g., ERP, CRM, KB repositories;
- Shared **memory** management for data and context sharing among agents;
- **Governance** layer, including explainability, privacy, security, safety guardrails, etc.





Given a user task, the goal of the agentic AI platform is to identify (compose) an agent (group of agents) capable to executing the given task. So the first component we need is a **reasoning** module capable of decomposing a task into sub-tasks, with execution of the respective agents orchestrated by an orchestration engine.

Chain of Thought (CoT) is the most widely used decomposition framework today to transform complex tasks into multiple manageable tasks and shed light into an interpretation of the model's thinking process. Further, the **ReAct** (reasoning and acting) framework allows an agent to critically evaluate its own actions and outputs, learn from them, and subsequently refine its plan / reasoning process.

Agent composition implies the existence of an agent marketplace / registry of agents — with a well-defined description of the agent capabilities and constraints. For example, the Agent2Agent (A2A) protocol specifies the notion of an Agent Card (a JSON document) that serves as a digital "business card" for agents. It includes the following key information:

Copy

Identity: name, description, provider information.

Service Endpoint: The url where the A2A service can be reached.

SKILLS: A LIST OF SPECIFIC TASKS OR FUNCTIONS THE AGENT CAN PERFORM (AgentSkills)

Given the need to orchestrate multiple agents, there is a need for a **system integration module** supporting different agent interaction patterns, e.g., agent-to-agent API, agent API providing output for human consumption, human triggering an AI agent, AI agent-to-agent with human in the Loop. The integration patterns need to be supported by the underlying Agent OS platform.

Andrew Ng recently talked about this aspect from a performance perspective:

Today, a lot of LLM output is for human consumption. But in an agentic workflow, an LLM might be prompted repeatedly to reflect on and improve its output, use tools, plan and execute multiple steps, or implement multiple agents that collaborate. So, we might generate hundreds of thousands of tokens or more before showing any output to a user. This makes fast token generation very desirable and makes slower generation a bottleneck to taking better advantage of existing models.

In this article, we refer to the model context protocol (MCP) proposed by Anthropic recently to connect AI agents to external systems / tools where

1. *Resources*: This is structured data that the server can give to the AI. For example, code snippets, parts of a document, or database query results; anything that adds factual context.
2. *Prompts*: are pre-made instructions or templates that the server can provide. Think of saved prompts for summarizing text or generating code in a specific style.
3. *Tools*: refer to actual actions the AI can ask the server to perform. On the retrieval side, these include querying a database, searching the web, etc.

By standardizing these, any AI system using **MCP** can understand how to request data (resources), provide instructions (prompts), or perform actions (tools) through any compatible MCP server.

Given the long-running nature of complex agents, memory management is key for agentic AI systems.

This entails both context sharing between tasks and maintaining execution context over long periods.

product search (MIPS). For fast retrieval, the approximate nearest neighbors (ANN) algorithm is used that returns approximately top k-nearest neighbors with an accuracy trade-off versus a huge speed gain.

Finally, the **governance** module. We need to ensure that data shared by the user specific to a task, or user profile data that cuts across tasks; is only shared with the relevant agents (table / report authentication and access control). Refer to my previous [article](#) on Responsible AI Agents for a discussion on the key dimensions needed to enable a well governed AI agent platform in terms of hallucination [guardrails](#), data quality, [privacy](#), reproducibility, explainability, human-in-the-loop ([HITL](#)), etc.

### 3. Agentic Discovery & Matchmaking

In this section, we focus on the discovery aspect of AI agents, that is, identifying agents / tools capable of executing a given user prompt (goal) — illustrated in Fig. 4.

This implies that there exists a marketplace (registry of agents), with a well-defined description of the agent capabilities and constraints.

Identifying the right agents / tools that can execute a given task, given the task requirements and capabilities / constraints of the agents / tools available in the marketplace.

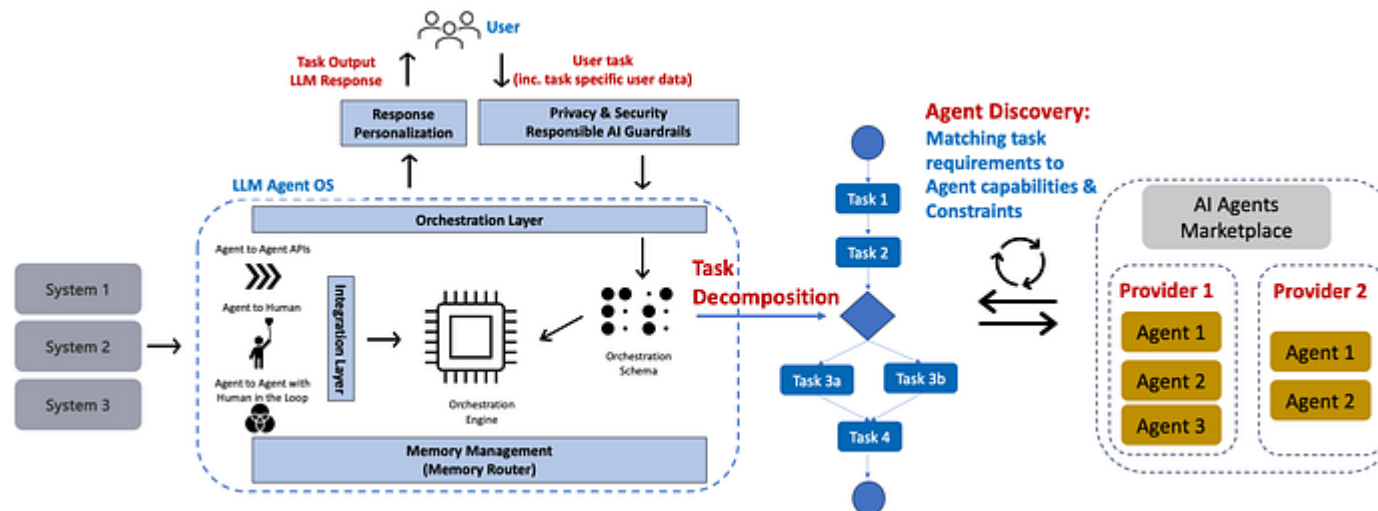


Fig. 4: Discovery and matchmaking for Agentic AI (Image by Author)

### 3.1 L2R algorithm for Agentic Discovery

In this section, we propose a learning-to-rank (L2R) algorithm to discover the right agent(s) corresponding to a user prompt. For example the available agents in the marketplace for the e-shopping scenario illustrated in Fig. 5 can be: "Check for price", "Compare products", "Deliver products", "Pay for order", etc.

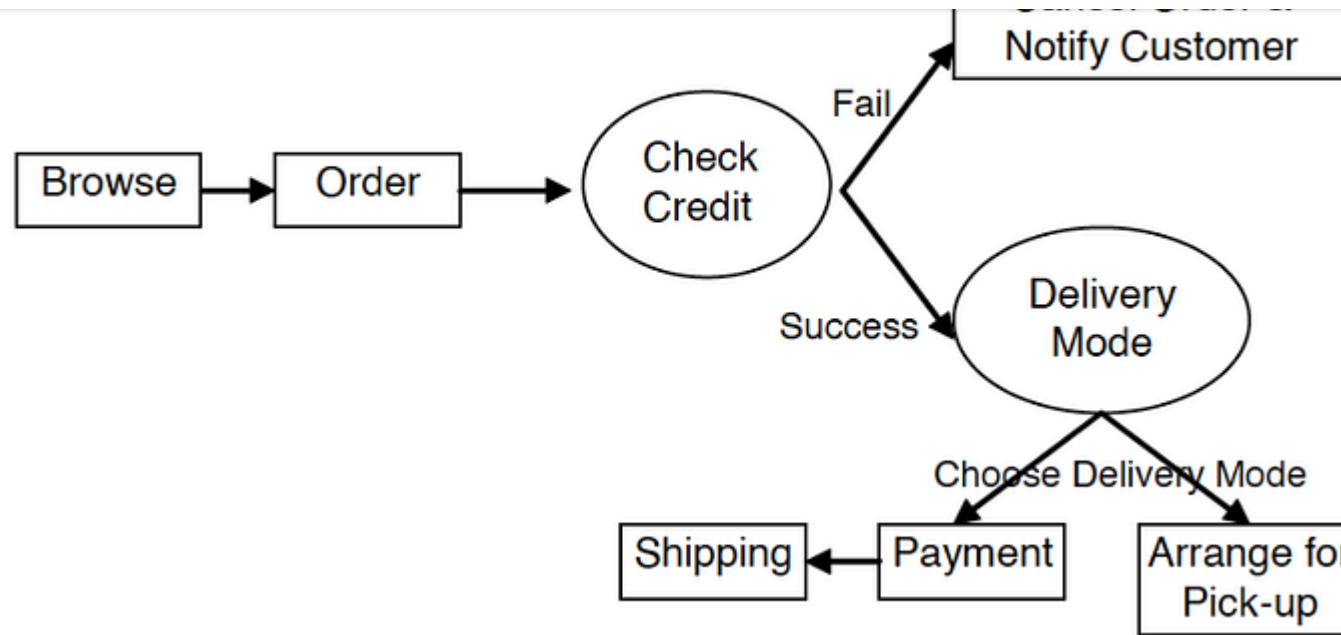


Fig. 5: E-shopping scenario with non-determinism (Image by Author)

Given that both the prompts and agent service descriptions are provided in natural language, this is primarily a multi-class **text classification** problem. Unfortunately, traditional vector or cosine similarity based algorithms face challenges here due to the following reasons:

- the agent descriptions are ambiguous and overlap with that of other agents (possibly provided by a different vendor);
- the user prompts (including those of different users) themselves can be ambiguous and overlapping;

The L2R algorithm to **select top-k agents** given a user prompt is illustrated in Fig. 6. We first convert agent (class) descriptions to semantic embeddings offline and use them to train the L2R model. The user prompts and the agents use the same generic embedding model. The inference results including the agent description embeddings during training and inferencing are cached to enable the meta-learning process for the L2R algorithm.

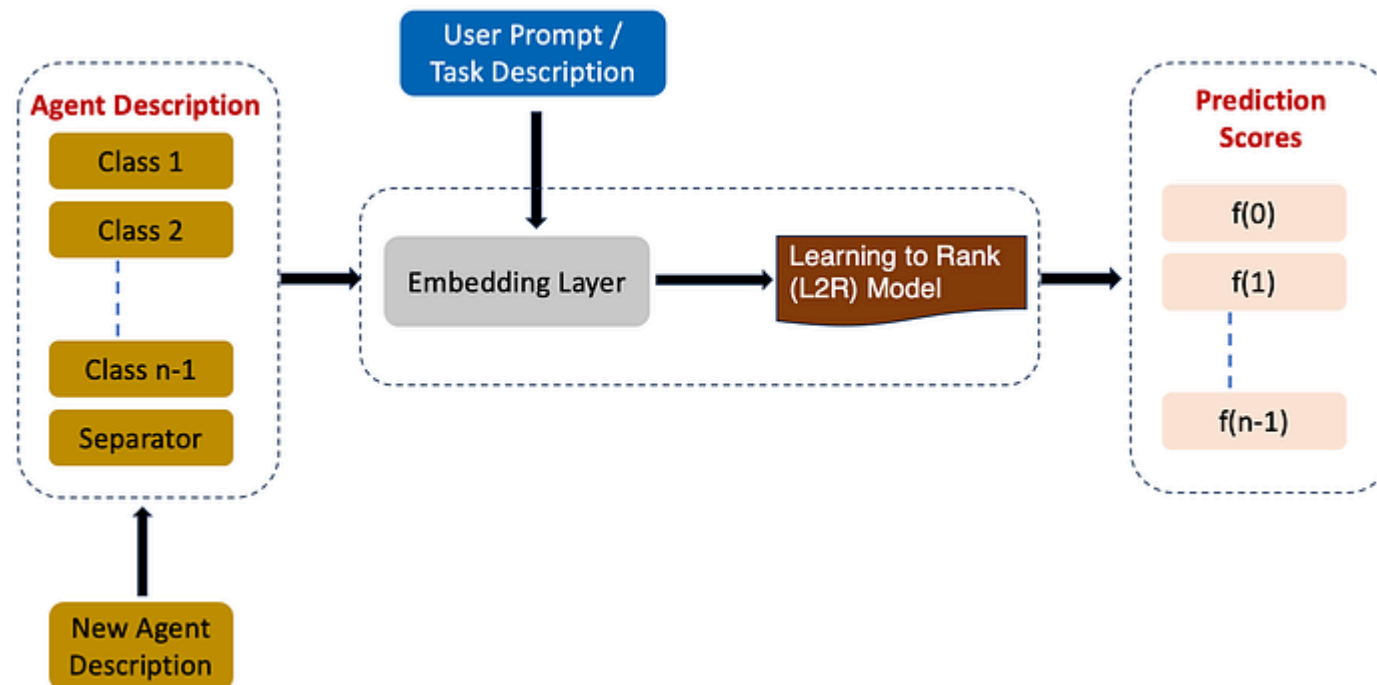


Fig. 6: Learning-to-Rank (L2R) algorithm to select top-k Agents given a User prompt (Image by Author)

## What is the price of Product X?

the ratings of agents "Check for Price", "List provider Y's products (Y is the provider of product X)", "Separator", "Compare Products", "Non-Y's Products" are {2, 1, 0, -1, -2}, where higher positive **ratings** indicate that the agent description is more related to the user prompt / (decomposed) task description.

The "Separator" class indicates a class that is not defined in the current model. The L2R model maps the **separator** class to the top-k position, i.e., the placeholder "Separator" class itself and agents whose service descriptions are not relevant to the user prompt / task description are ranked below this separator.

Finally, one can also challenge as to why not pose the classification problem itself to an LLM at run-time?

Indeed, recent studies have shown promising results with respect to LLM based **few-shot** text classification. However, in our experiments, we found that due to hallucinations, the LLM's outputs often did not contain the defined agent descriptions. Also, it was becoming a challenge to fit all the task and agent descriptions, with few-shot examples, into an LLM invocation given the cost and



### 3.2 Constraints Model for Agentic Discovery

In this section, we outline a constraints based formalism to overcome the limitations of textual descriptions.

*For example*, let us consider a house painting agent  $C$  whose services can be reserved online (via credit card). Given this, the fact that the user requires a valid credit card is a constraint, and the fact that the user's house will be painted within a certain timeframe are its capabilities. In addition, we also need to consider any constraints of  $C$  during the actual execution phase, e.g., the fact that  $C$  can only provide the service on weekdays (and not on weekends). The above restriction might be a problem if the user specified task requires getting the work done during weekends.

In general, constraints refer to the conditions that need to be satisfied to initiate an execution and capabilities reflect the expected outcome after the execution terminates — illustrated in Fig. 7 for a travel booking scenario.



---

Refer to my ICAART 2024 [paper](#) for the full constraints based agentic formalism and a detailed discussion on the same.

## 4. MCP Tools Governance: Best Practices & Guidelines

Given the above discussion reg. challenges associated with ungoverned MCP tools in the enterprise, we provide some best practices and guidelines to address them in this section:

- Consider the tools, prompts, resources, etc. capabilities published by an MCP server, and not only the tool descriptions during discovery. OpenAI's [recommendation](#) here is to:

Include examples and edge cases, especially to rectify any recurring failures. (Note: Adding examples may hurt performance for reasoning models).

- Tools are generally invoked as part of an agent-to-agent (A2A) interaction, where the output of a tool (invoked by one agent) goes as input **context** to the next agent. Unfortunately, the MCP specification today does not provide any provision to specify the **count of tokens** produced by a tool. Until this happens, the recommendation is for agents to implement their own context management strategy.

servers have overlapping tool names (as described above) — then disambiguation becomes difficult. OpenAI Agents SDK would raise an error in this case, while Claude Code would prefix tool names with unique identifiers to work around this issue.

The recommendation here is to have **hierarchical** namespaces of thematically related tools, e.g., GitHub MCP server's dynamic tool discovery.

The goal is to enable hierarchical tool-calling, with (1) selecting a category, (2) followed by selecting a tool.

## 5. Conclusion

In this article, we focused on the challenges of uncontrolled adoption of agents and tools in the enterprise. Unfortunately, deploying agents and tools in an ungoverned fashion only makes the situation worse — leading to a loss in productivity.

We highlighted the current challenges of leveraging LLMs as an execution engine for Agentic AI systems. We showed that to execute a complex task, a pre-requisite

based on the textual descriptions, and a more expressive constraints based formalism to specify and discover top-k agents given a user prompt / task.

We finally analyzed the current MCP specification in detail, providing best practices and recommendations to enable governed usage of MCP tools in the enterprise.

[#agentic-ai](#) [#agentic-ai-architecture](#) [#mcp-server](#) [#ai-governance](#) [#ai-agent-discovery](#)